

COMPUTER VISION

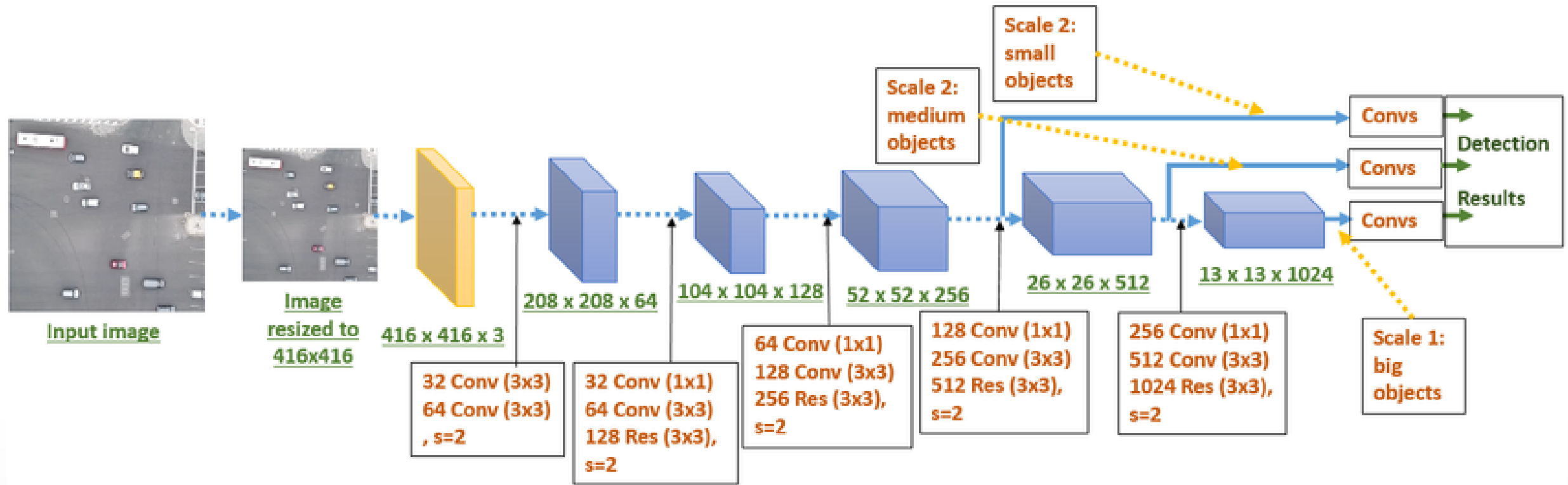
Object Detection; Face Detection

Dr. Sridhar Pappu

30/OCT/2025

Object Detection with YOLOv3

YOLOv3 Architecture



Summary Architecture:

- Backbone (Darknet-53): 53 conv layers (organized in residual blocks).
- Whole detector: 75 conv layers (including the 53 in backbone) with weights + non-parametric ops (route/shortcut/upsample) + 3 YOLO detection layers → indices end at 82, 94, 106, hence the common “106-layer” count.

Create and Save Model

- Download pre-trained model weights, trained using DarkNet-53 on the COCO dataset.
- Define a Keras model that has the right number and type of layers to match the downloaded model weights. The model architecture is called DarkNet and was originally loosely based on the VGG-16 model.
- Load the weights, pass in the defined Keras model and set the weights into the layer.
- Save the model for future use.

```
def make_yolov3_model():
    input_image = Input(shape=(None, None, 3))

    # Layer 0 => 4
    x = _conv_block(input_image, [{'filter': 32, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 0},
                                  {'filter': 64, 'kernel': 3, 'stride': 2, 'bnorm': True, 'leaky': True, 'layer_idx': 1},
                                  {'filter': 32, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 2},
                                  {'filter': 64, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 3}])

    # Layer 5 => 8
    x = _conv_block(x, [{'filter': 128, 'kernel': 3, 'stride': 2, 'bnorm': True, 'leaky': True, 'layer_idx': 5},
                        {'filter': 64, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 6},
                        {'filter': 128, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 7}])

    # Layer 9 => 11
    x = _conv_block(x, [{'filter': 64, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 9},
                        {'filter': 128, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 10}])

    # Layer 12 => 15
    x = _conv_block(x, [{'filter': 256, 'kernel': 3, 'stride': 2, 'bnorm': True, 'leaky': True, 'layer_idx': 12},
                        {'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 13},
                        {'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 14}])

    # Layer 16 => 36
    for i in range(7):
        x = _conv_block(x, [{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 16+i*3},
                            {'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 17+i*3}])

    skip_36 = x
    # Layer 37 => 40
    x = _conv_block(x, [{'filter': 512, 'kernel': 3, 'stride': 2, 'bnorm': True, 'leaky': True, 'layer_idx': 37},
                        {'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 38},
                        {'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 39}])

    # Layer 41 => 61
    for i in range(7):
        x = _conv_block(x, [{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 41+i*3},
                            {'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 42+i*3}])

    skip_61 = x
    # Layer 62 => 65
    x = _conv_block(x, [{'filter': 1024, 'kernel': 3, 'stride': 2, 'bnorm': True, 'leaky': True, 'layer_idx': 62},
                        {'filter': 512, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 63},
                        {'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 64}])

    # Layer 66 => 74
    for i in range(3):
        x = _conv_block(x, [{'filter': 512, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 66+i*3},
                            {'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 67+i*3}])

    # Layer 75 => 79
    x = _conv_block(x, [{'filter': 512, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 75},
                        {'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 76},
                        {'filter': 512, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 77},
                        {'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 78},
                        {'filter': 512, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 79}], skip=False)

    # Layer 80 => 82
    yolo_82 = _conv_block(x, [{'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 80},
                              {'filter': 255, 'kernel': 1, 'stride': 1, 'bnorm': False, 'leaky': False, 'layer_idx': 81}], skip=False)

    # Layer 83 => 86
    x = _conv_block(x, [{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 84}], skip=False)
    x = UpSampling2D(2)(x)
    x = concatenate([x, skip_61])

    # Layer 87 => 91
    x = _conv_block(x, [{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 87},
                        {'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 88},
                        {'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 89},
                        {'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 90},
                        {'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 91}], skip=False)

    # Layer 92 => 94
    yolo_94 = _conv_block(x, [{'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 92},
                              {'filter': 255, 'kernel': 1, 'stride': 1, 'bnorm': False, 'leaky': False, 'layer_idx': 93}], skip=False)

    # Layer 95 => 98
    x = _conv_block(x, [{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 96}], skip=False)
    x = UpSampling2D(2)(x)
    x = concatenate([x, skip_36])

    # Layer 99 => 106
    yolo_106 = _conv_block(x, [{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 99},
                              {'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 100},
                              {'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 101},
                              {'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 102},
                              {'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 103},
                              {'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 104},
                              {'filter': 255, 'kernel': 1, 'stride': 1, 'bnorm': False, 'leaky': False, 'layer_idx': 105}], skip=False)

    model = Model(input_image, [yolo_82, yolo_94, yolo_106])
    return model
```

Make a Prediction

- The model expects input images to be colour and square shape of 416 x 416 px.
- Running the example returns a list of three NumPy arrays, the shape of which is displayed as output. These arrays predict both the bounding boxes and class labels but are encoded. They must be interpreted.

```
# Load yolov3 model
model = load_model('yolov3.keras', compile=False)
# define the expected input shape for the model
input_w, input_h = 416, 416
# define our new photo
photo_filename = '../Data/zebra.jpg'
# Load and prepare image
image, image_w, image_h = load_image_pixels(photo_filename, (input_w, input_h))
# make prediction
yhat = model.predict(image)
# summarize the shape of the list of arrays
print([a.shape for a in yhat])

1/1 ————— 2s 2s/step
[(1, 13, 13, 255), (1, 26, 26, 255), (1, 52, 52, 255)]
```

- 1 → batch size (inference run on 1 image).
- (13, 13), (26, 26), (52, 52) → grid sizes at the three detection scales (downsampling factors 32, 16, 8 of the input).
- 255 → channels per grid cell = (anchors per scale) × (5 + number_of_classes). For COCO: 3 anchors × (4 bbox coords + 1 objectness + 80 class scores) = 3 × 85 = 255.

Make a Prediction and Interpret Result

- The function will take each one of the NumPy arrays, one at a time, and decode the candidate bounding boxes and class predictions.

```
def decode_netout(netout, anchors, obj_thresh, net_h, net_w):
    grid_h, grid_w = netout.shape[:2]
    nb_box = 3
    netout = netout.reshape((grid_h, grid_w, nb_box, -1))
    nb_class = netout.shape[-1] - 5
    boxes = []
    netout[..., :2] = _sigmoid(netout[..., :2])
    netout[..., 4:] = _sigmoid(netout[..., 4:])
    netout[..., 5:] = netout[..., 4][..., np.newaxis] * netout[..., 5:]
    netout[..., 5:] *= netout[..., 5:] > obj_thresh

    for i in range(grid_h*grid_w):
        row = i / grid_w
        col = i % grid_w
        for b in range(nb_box):
            # 4th element is objectness score
            objectness = netout[int(row)][int(col)][b][4]
            if(objectness.all() <= obj_thresh): continue
            # first 4 elements are x, y, w, and h
            x, y, w, h = netout[int(row)][int(col)][b][:4]
            x = (col + x) / grid_w # center position, unit: image width
            y = (row + y) / grid_h # center position, unit: image height
            w = anchors[2 * b + 0] * np.exp(w) / net_w # unit: image width
            h = anchors[2 * b + 1] * np.exp(h) / net_h # unit: image height
            # Last elements are class probabilities
            classes = netout[int(row)][col][b][5:]
            box = BoundBox(x-w/2, y-h/2, x+w/2, y+h/2, objectness, classes)
            boxes.append(box)

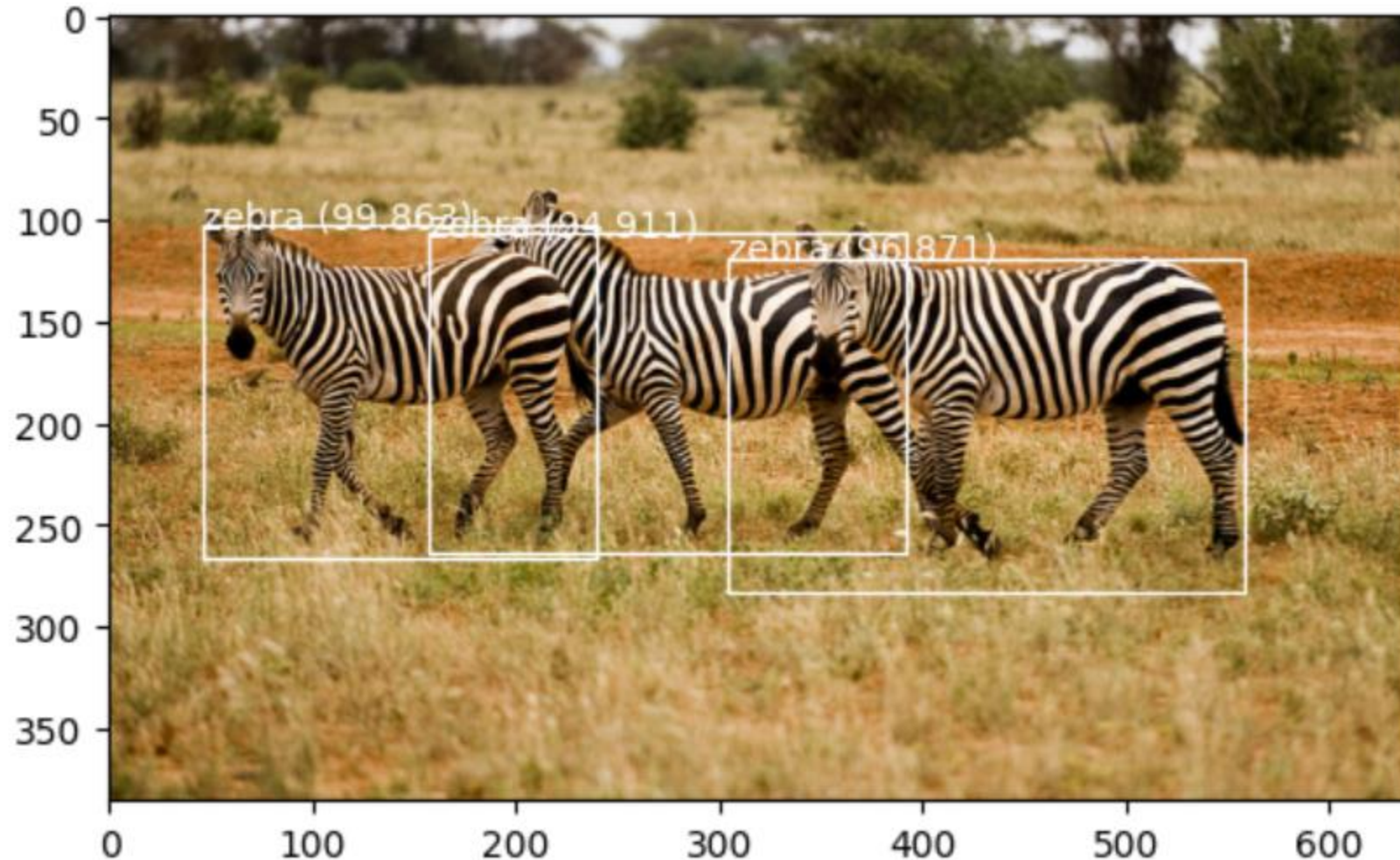
    return boxes
```


Make a Prediction and Interpret Result

zebra 94.91055011749268

zebra 99.86327290534973

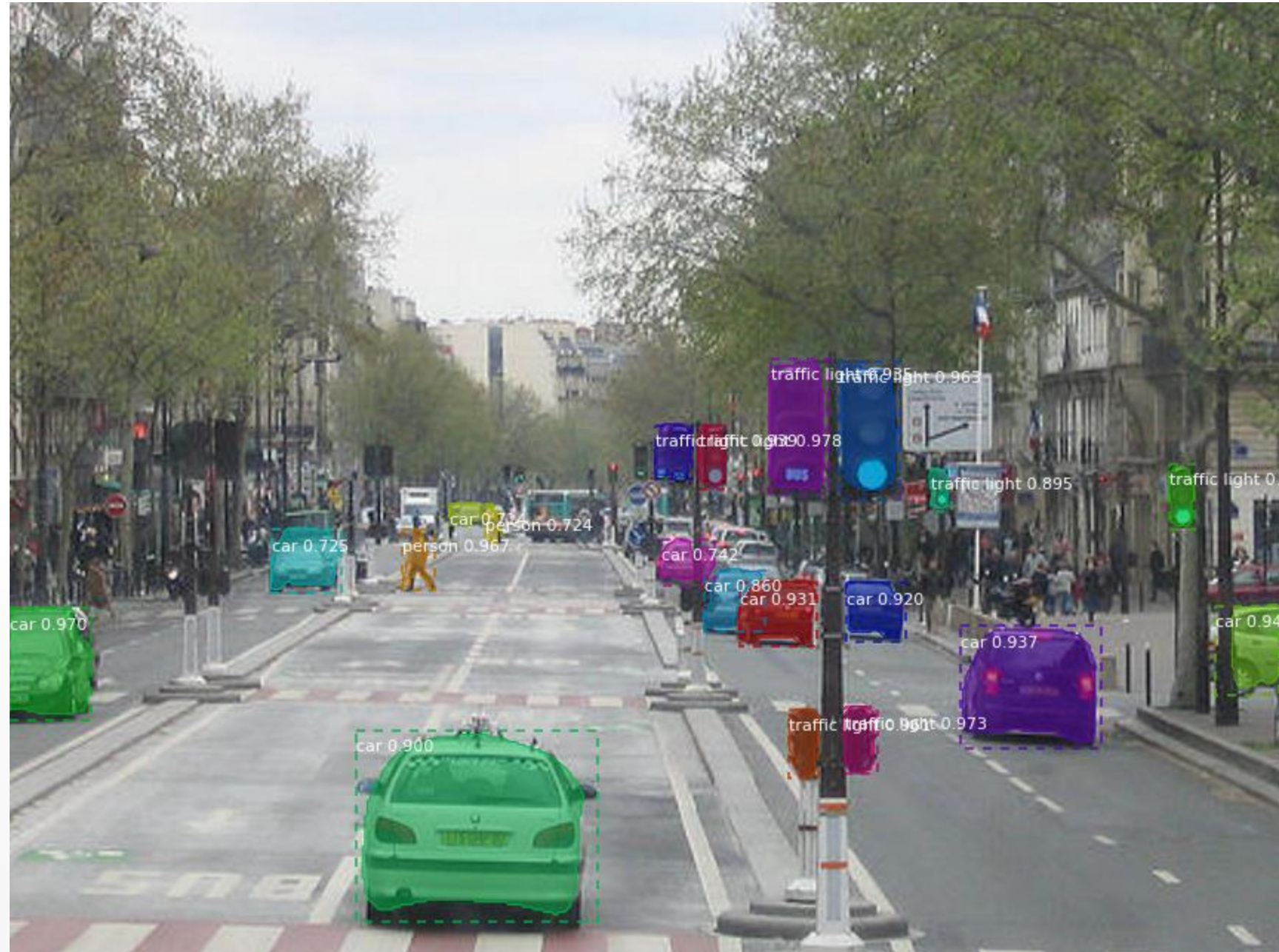
zebra 96.8708872795105



Object Detection with Mask R-CNN

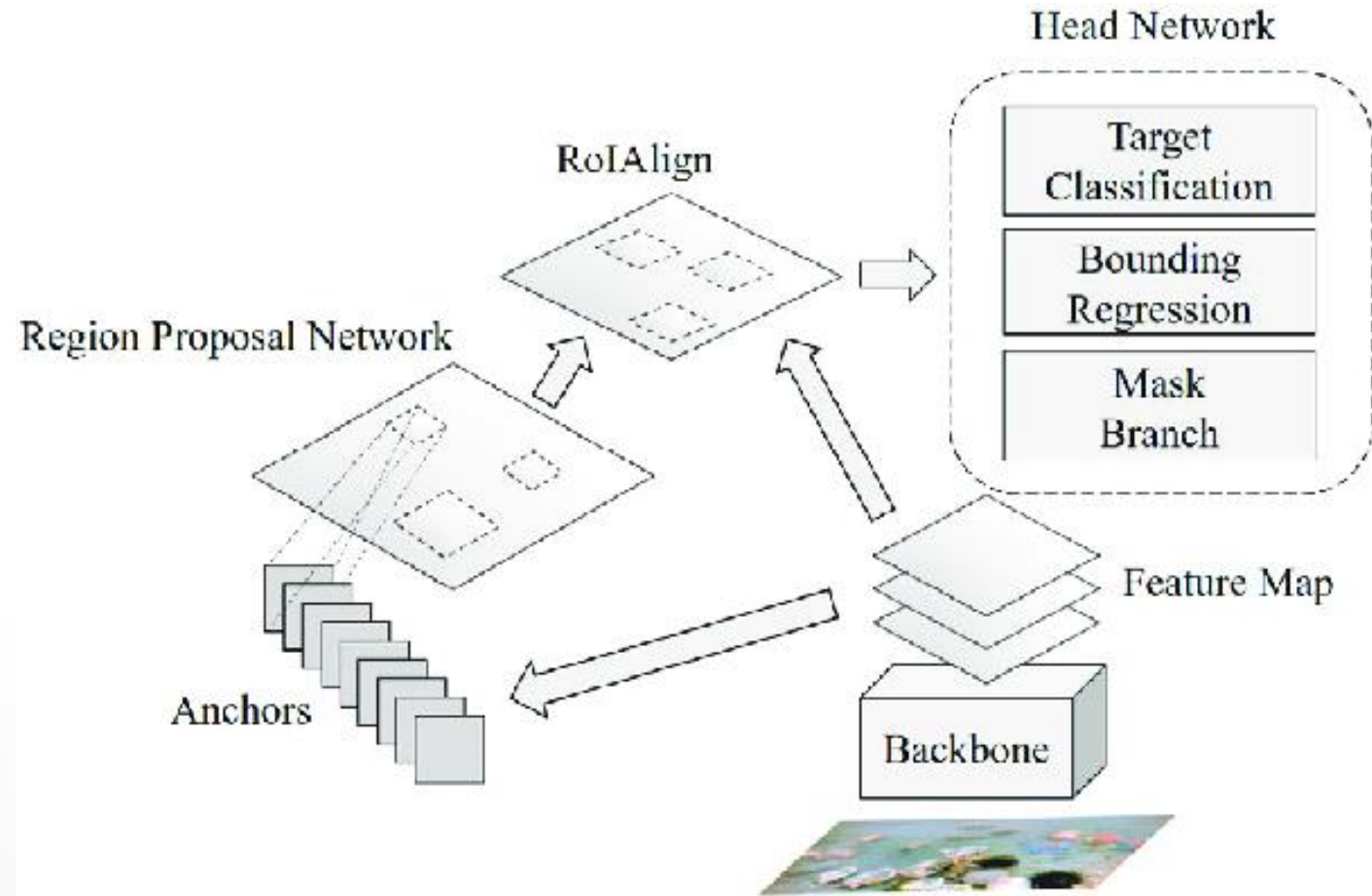
Mask R-CNN

- An extension of object detection involves marking the specific pixels in the image that belong to each detected object instead of using coarse bounding boxes during object localization.
- This harder version of the problem is generally referred to as object segmentation or semantic segmentation.
- It extends the Faster R-CNN architecture by adding a branch for predicting segmentation masks, enabling pixel-level precision for each detected object.



Mask R-CNN Architecture

- **Backbone Network:** Typically, ResNet for feature extraction.
- **Region Proposal Network (RPN):** Suggests candidate object regions.
- **RoIAlign:** Converts variable-sized regions into fixed-size feature maps for consistent processing.
- **Mask Representation:** Outputs a binary mask for each object instance, maintaining spatial correspondence.

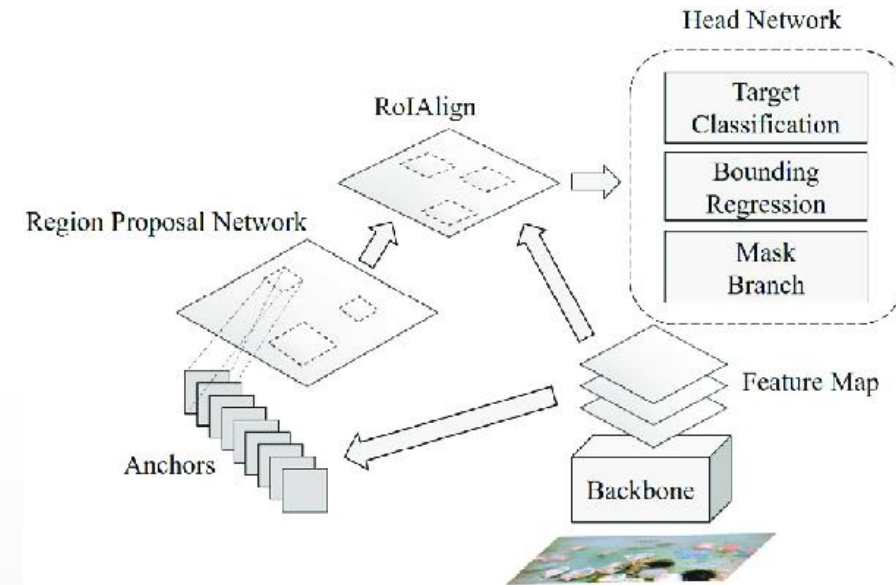


How Mask R-CNN Works

Mask R-CNN operates in two main stages:

- **Region Proposal Network (RPN):** This stage generates region proposals likely to contain objects. These proposals are refined to ensure accurate bounding boxes.
- **Parallel Branches:** One branch classifies objects and refines bounding boxes. Another branch predicts binary masks for each region, providing pixel-level segmentation.

The key innovation is the mask branch, which uses a fully convolutional network to predict segmentation masks. To ensure precise alignment between regions and masks, Mask R-CNN employs RoIAlign, which improves upon RoIPooling by eliminating quantization errors.



Mask R-CNN Advantages

ADVANTAGES

- **Precise Instance Segmentation:** Provides accurate pixel-level masks for each object.
- **Versatility:** Handles object detection, classification, and segmentation in a unified framework.
- **State-of-the-Art Performance:** Excels in benchmarks for instance segmentation.

LIMITATIONS

- **Computational Intensity:** Requires significant computational resources, including GPUs.
- **Real-Time Constraints:** Limited suitability for real-time applications due to its relatively low inference speed (~2 FPS).

Mask R-CNN APPLICATIONS

Mask R-CNN is widely used in various domains, including:

- Medical Imaging: Tumor detection and segmentation.
- Autonomous Vehicles: Object detection and scene understanding.
- Drone Mapping: Identifying features in satellite imagery

Matterport Mask R-CNN Project

- There are perhaps three main use cases for using the Mask R-CNN model with the Matterport library; they are:
 - Object Detection Application: Use a pre-trained model for object detection on new images.
 - New Model via Transfer Learning: Use a pre-trained model as a starting point in developing a model for a new object detection dataset.
 - New Model from Scratch: Develop a new model from scratch for an object detection dataset.
- The best of breed third-party implementations of Mask R-CNN is the Mask R-CNN Project developed by Matterport. The project is open source released under a permissive license (i.e., MIT license) and the code has been widely used on a variety of projects and Kaggle competitions.

Object Detection with Mask R-CNN

- We will use the Matterport Mask R-CNN library to perform object detection on arbitrary photographs. Much like using a pre-trained deep CNN for image classification, such as VGG-16 trained on an ImageNet dataset, we can use a pre-trained Mask R-CNN model to detect objects in new photographs. In this case, we will use a Mask R-CNN trained on the MSCOCO object detection problem.

Mask R-CNN Installation

- Step 1: Clone the Mask R-CNN GitHub repository
 - From the command line, run `git clone https://github.com/matterport/Mask_RCNN.git`
- Step 2: Install the Mask R-CNN library
 - From the command line, type:
 - `cd Mask_RCNN python`
 - `setup.py install`
- Step 3: Confirm the library was installed
 - `pip show mask-rcnn`
- Step 4: Download model weights
 - <https://goo.gl/a2p7dS>

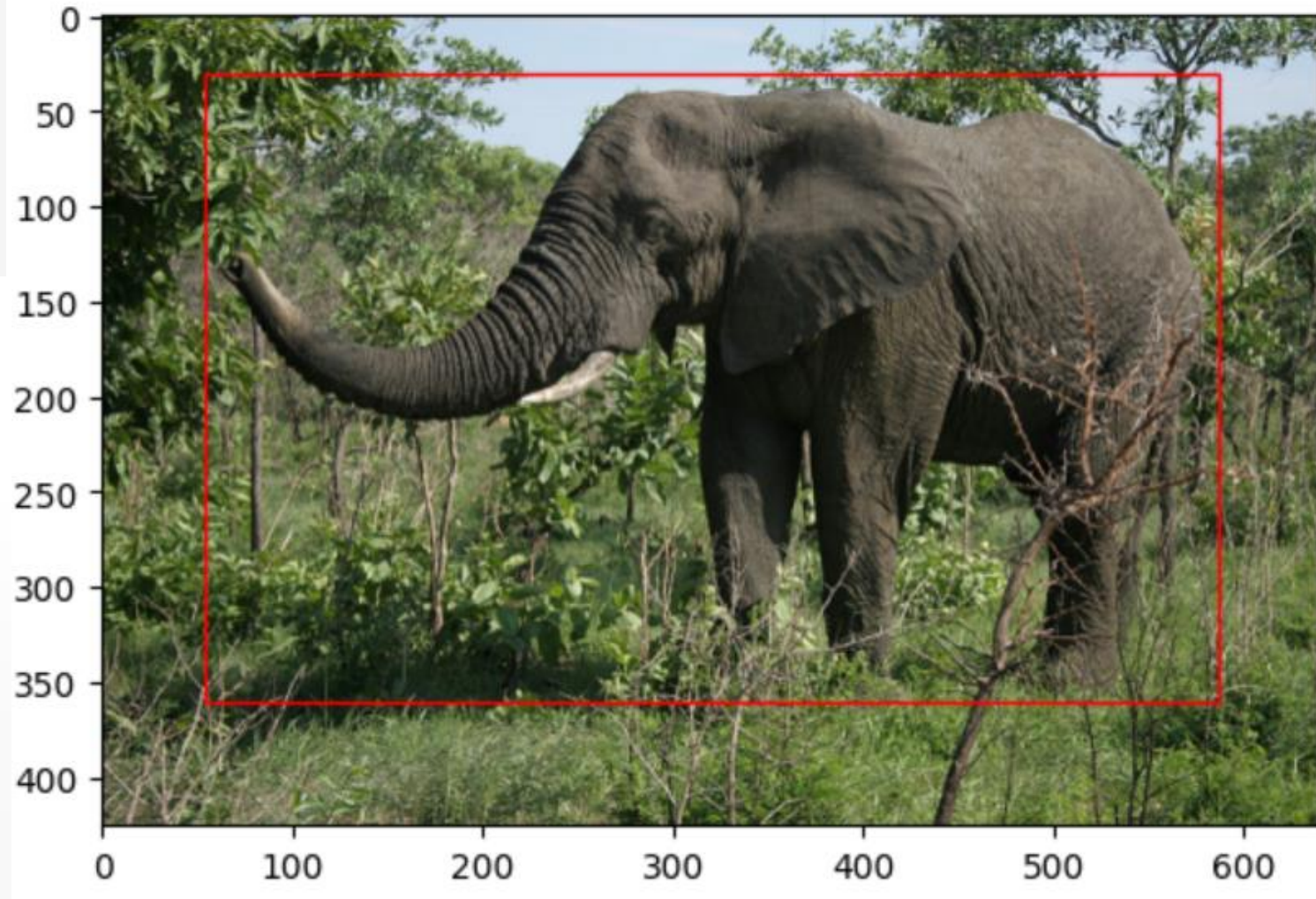
Load Model and Make Predictions

- Step 1: Clone the Mask R-CNN GitHub repository
 - From the command line, run `git clone https://github.com/matterport/Mask_RCNN.git`
- Step 2: Install the Mask R-CNN library
 - From the command line, type:
 - `cd Mask_RCNN python`
 - `setup.py install`
- Step 3: Confirm the library was installed
 - `pip show mask-rcnn`
- Step 4: Download model weights
 - <https://goo.gl/a2p7dS>

Load Model and Make Predictions

```
# define the model
rcnn = MaskRCNN(mode='inference', model_dir='.', config=TestConfig())
# load coco model weights
rcnn.load_weights('../Data/mask_rcnn_coco.h5', by_name=True)
# load photograph
img = load_img('../Data/elephant.jpg')
img = img_to_array(img)
# make prediction
results = rcnn.detect([img], verbose=0)
# visualize the results
draw_image_with_boxes('../Data/elephant.jpg', results[0]['rois'])
```

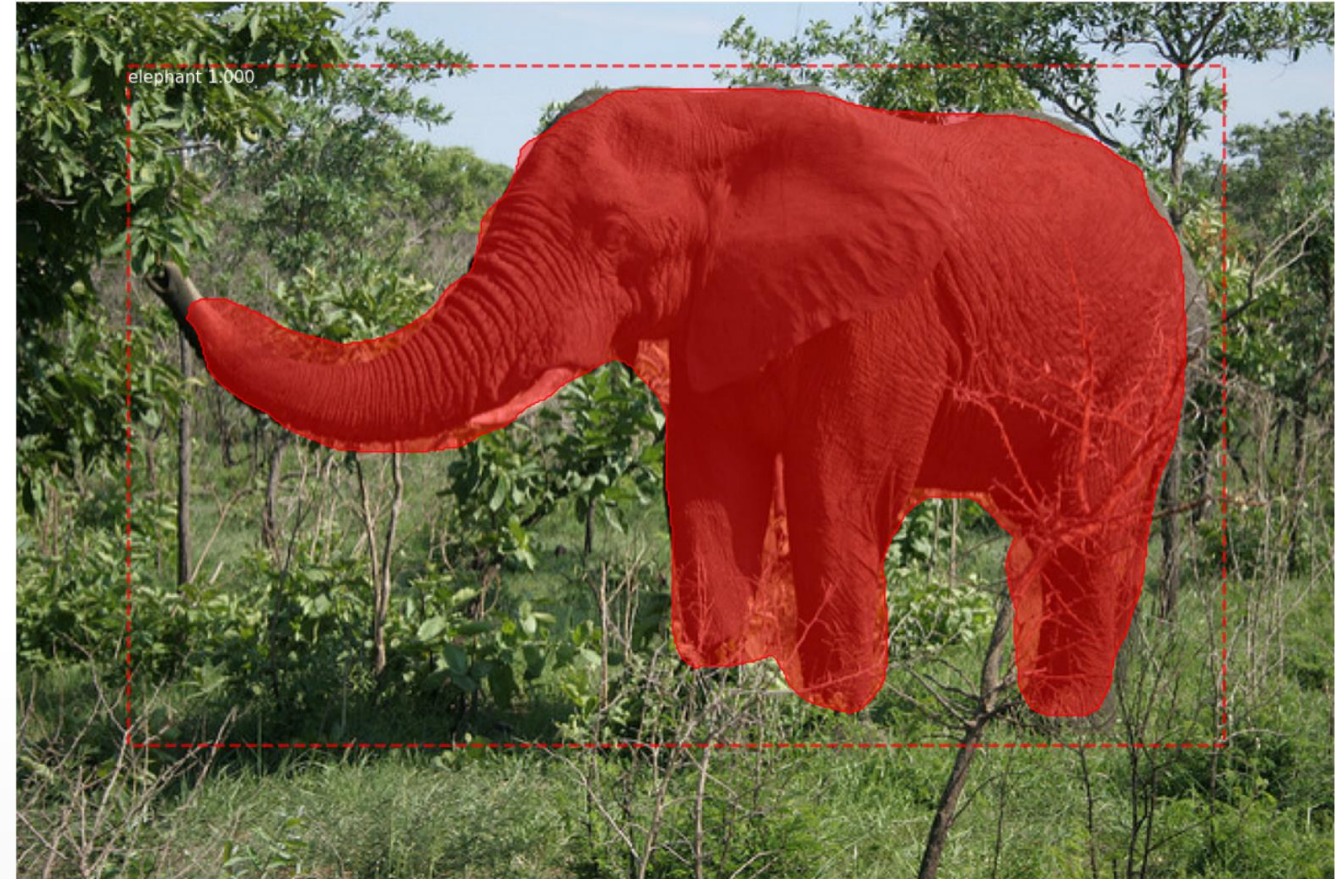
- Running the example loads the model and performs object detection. More accurately, we have performed **object localization**, only drawing bounding boxes around detected objects.
 - Note the difference from detection where we have bounding boxes that are detecting objects within.
- In this case, we can see that the model has correctly located the single object in the photo, the elephant, and drawn a red box around it.



Example of Object Detection

- Now that we know how to load the model and use it to make a prediction, let's update the example to perform real object detection.
- That is, in addition to localizing objects, we want to know what they are.
- The Mask RCNN API provides a function called `display_instances()` that will take the array of pixel values for the loaded image and the aspects of the prediction dictionary, such as the bounding boxes, scores, and class labels, and will plot the photo with all of these annotations.

```
# define the model
rcnn = MaskRCNN(mode='inference', model_dir='.', config=TestConfig())
# load coco model weights
rcnn.keras_model.load_weights('../Data/mask_rcnn_coco.h5', by_name=True)
# load photograph
img = load_img('../Data/elephant.jpg')
img = img_to_array(img)
# make prediction
results = rcnn.detect([img], verbose=0)
# get dictionary for first prediction
r = results[0]
# show photo with bounding boxes, masks, class labels and scores
display_instances(img, r['rois'], r['masks'], r['class_ids'], class_names, r['scores'])
```



- Bounding Box: Dotted bounding box around each detected object.
- Class Label: Class label assigned each detected object written in the top left corner of the bounding box.
- Prediction Confidence: Confidence of class label prediction for each detected object written in the top left corner of the bounding box.
- Object Mask Outline: Polygon outline for the mask of each detected object.
- Object Mask: Polygon fill for the mask of each detected object.

Classes in COCO Dataset

define 81 classes that the coco model knows about

```
class_names = ['BG', 'person', 'bicycle', 'car', 'motorcycle', 'airplane',  
               'bus', 'train', 'truck', 'boat', 'traffic light',  
               'fire hydrant', 'stop sign', 'parking meter', 'bench', 'bird',  
               'cat', 'dog', 'horse', 'sheep', 'cow', 'elephant', 'bear',  
               'zebra', 'giraffe', 'backpack', 'umbrella', 'handbag', 'tie',  
               'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball',  
               'kite', 'baseball bat', 'baseball glove', 'skateboard',  
               'surfboard', 'tennis racket', 'bottle', 'wine glass', 'cup',  
               'fork', 'knife', 'spoon', 'bowl', 'banana', 'apple',  
               'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza',  
               'donut', 'cake', 'chair', 'couch', 'potted plant', 'bed',  
               'dining table', 'toilet', 'tv', 'laptop', 'mouse', 'remote',  
               'keyboard', 'cell phone', 'microwave', 'oven', 'toaster',  
               'sink', 'refrigerator', 'book', 'clock', 'vase', 'scissors',  
               'teddy bear', 'hair drier', 'toothbrush']
```

Let Us Test a Random Image from the Internet

```
# define 81 classes that the coco model knows about
class_names = ['BG', 'person', 'bicycle', 'car', 'motorcycle', 'airplane',
               'bus', 'train', 'truck', 'boat', 'traffic light',
               'fire hydrant', 'stop sign', 'parking meter', 'bench', 'bird',
               'cat', 'dog', 'horse', 'sheep', 'cow', 'elephant', 'bear',
               'zebra', 'giraffe', 'backpack', 'umbrella', 'handbag', 'tie',
               'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball',
               'kite', 'baseball bat', 'baseball glove', 'skateboard',
               'surfboard', 'tennis racket', 'bottle', 'wine glass', 'cup',
               'fork', 'knife', 'spoon', 'bowl', 'banana', 'apple',
               'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza',
               'donut', 'cake', 'chair', 'couch', 'potted plant', 'bed',
               'dining table', 'toilet', 'tv', 'laptop', 'mouse', 'remote',
               'keyboard', 'cell phone', 'microwave', 'oven', 'toaster',
               'sink', 'refrigerator', 'book', 'clock', 'vase', 'scissors',
               'teddy bear', 'hair drier', 'toothbrush']
```

Object Detection with Mask R-CNN

How to Develop a New Object Detection Model

Plot With Object Detection Mask Overlaid

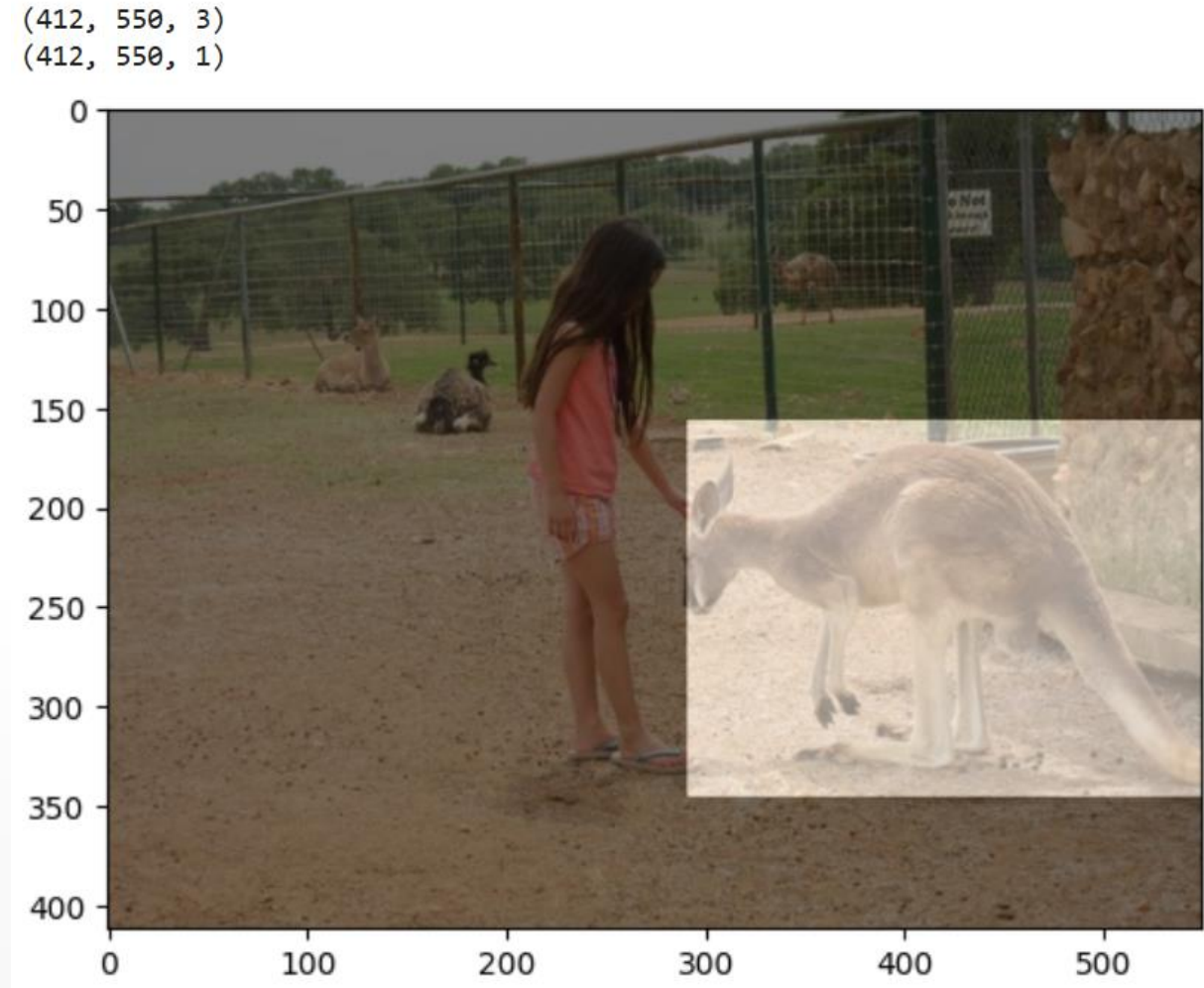
```
# train set
train_set = KangarooDataset()
train_set.load_dataset('../Data/kangaroo', is_train=True)
train_set.prepare()
# Load an image
image_id = 2
image = train_set.load_image(image_id)
print(image.shape)
# Load image mask
mask, class_ids = train_set.load_mask(image_id)
print(mask.shape)
# plot image
pyplot.imshow(image)
# plot mask
pyplot.imshow(mask[:, :, 0], cmap='gray', alpha=0.5)
pyplot.show()
```

(288, 460, 3)
(288, 460, 1)



Plot With Object Detection Mask Overlaid

```
# train set
train_set = KangarooDataset()
train_set.load_dataset('../Data/kangaroo', is_train=True)
train_set.prepare()
# Load an image
image_id = 10
image = train_set.load_image(image_id)
print(image.shape)
# Load image mask
mask, class_ids = train_set.load_mask(image_id)
print(mask.shape)
# plot image
pyplot.imshow(image)
# plot mask
pyplot.imshow(mask[:, :, 0], cmap='gray', alpha=0.5)
pyplot.show()
```



Plot Multiple Photos With Mask

```
# train set
train_set = KangarooDataset()
train_set.load_dataset('../Data/kangaroo', is_train=True)
train_set.prepare()
# Load an image
image_id = 0
image = train_set.load_image(image_id)
print(image.shape)
# Load image mask
mask, class_ids = train_set.load_mask(image_id)
print(mask.shape)
# plot first few images
for i in range(9):
    # define subplot
    pyplot.subplot(330 + 1 + i)
    # turn off axis labels
    pyplot.axis('off')
    # plot raw pixel data
    image = train_set.load_image(i)
    pyplot.imshow(image)
    # plot all masks
    mask, _ = train_set.load_mask(i)
    for j in range(mask.shape[2]):
        pyplot.imshow(mask[:, :, j], cmap='gray', alpha=0.3)
# show the figure
pyplot.show()
```

(319, 450, 3)

(319, 450, 2)



Plot Photo and Mask Using Built-In Function

```
# train set
train_set = KangarooDataset()
train_set.load_dataset('../Data/kangaroo', is_train=True)
train_set.prepare()
# define image id
image_id = 1
# load the image
image = train_set.load_image(image_id)
# load the masks and the class ids
mask, class_ids = train_set.load_mask(image_id)
# extract bounding boxes from the masks
bbox = extract_bboxes(mask)
# display image with masks and bounding boxes
display_instances(image, bbox, mask, class_ids, train_set.class_names)
```



Plot Photo and Mask Using Built-In Function

```
# train set
train_set = KangarooDataset()
train_set.load_dataset('../Data/kangaroo', is_train=True)
train_set.prepare()
# define image id
image_id = 24
# load the image
image = train_set.load_image(image_id)
# load the masks and the class ids
mask, class_ids = train_set.load_mask(image_id)
# extract bounding boxes from the masks
bbox = extract_bboxes(mask)
# display image with masks and bounding boxes
display_instances(image, bbox, mask, class_ids, train_set.class_names)
```



Plot Photo and Mask Using Built-In Function

```
# train set
train_set = KangarooDataset()
train_set.load_dataset('../Data/kangaroo', is_train=True)
train_set.prepare()
# define image id
image_id = 36
# Load the image
image = train_set.load_image(image_id)
# Load the masks and the class ids
mask, class_ids = train_set.load_mask(image_id)
# extract bounding boxes from the masks
bbox = extract_bboxes(mask)
# display image with masks and bounding boxes
display_instances(image, bbox, mask, class_ids, train_set.class_names)
```



Face Recognition

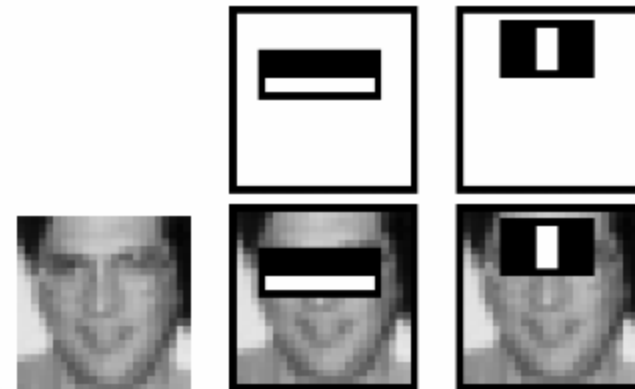
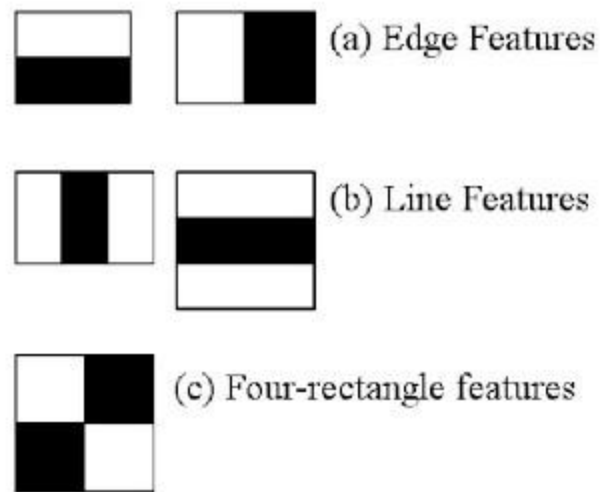
Process of Automatic Face Recognition

Face recognition is often described as a process that first involves four steps: face detection, face alignment, feature extraction, and finally face recognition.

- **Face Detection:** Problem in computer vision of **locating** and **localizing** one or more faces in a photograph.
 - Locating a face in a photograph refers to finding the coordinates of the face in the image, whereas
 - localization refers to demarcating the extent of the face, often via a bounding box around the face.
- **Face Alignment:** Normalize the face to be consistent with the database, such as geometry and photometrics.
- **Feature Extraction:** Extract features from the face that can be used for the recognition task.
- **Face Recognition:** Perform matching of the face against one or more known faces in a prepared database.

Face Detection Using Cascade Classifiers

- Object Detection using Haar feature-based cascade classifiers is a machine learning based approach where a cascade function is trained from a lot of positive and negative images. It is then used to detect objects in other images.
- Initially, the algorithm needs a lot of positive images (images of faces) and negative images (images without faces) to train the classifier. Then we need to extract features from it. For this, Haar features shown in the below image are used. They are just like our convolutional kernel.



- For more details, read https://docs.opencv.org/3.4/db/d28/tutorial_cascade_classifier.html.

Face Detection Using Cascade Classifiers

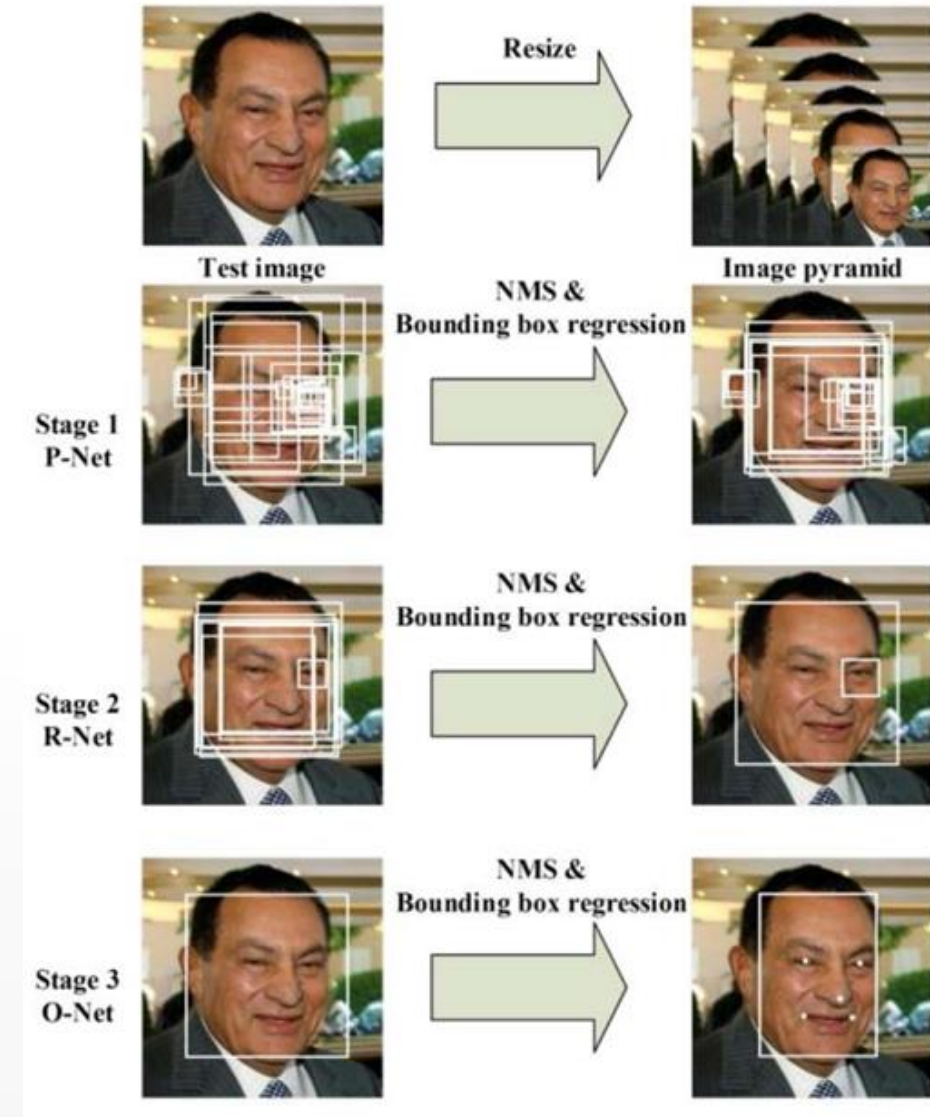
```
# plot photo with detected faces using opencv cascade classifier
from cv2 import imread
from cv2 import imshow
from cv2 import waitKey
from cv2 import destroyAllWindows
from cv2 import CascadeClassifier
from cv2 import rectangle
# Load the photograph
pixels = imread('../Data/face_detection_college_students.jpg')
# Load the pre-trained model
classifier = CascadeClassifier('../Data/haarcascade_frontalface_default.xml')
# perform face detection
bboxes = classifier.detectMultiScale(pixels)
# print bounding box for each detected face
for box in bboxes:
    # extract
    x, y, width, height = box
    x2, y2 = x + width, y + height
    # draw a rectangle over the pixels
    rectangle(pixels, (x, y), (x2, y2), (0,0,255), 1)
# show the image
imshow('face detection', pixels)
# keep the window open until we press a key
waitKey(0)
# close the window
destroyAllWindows()
```



Deep Learning for Face Recognition

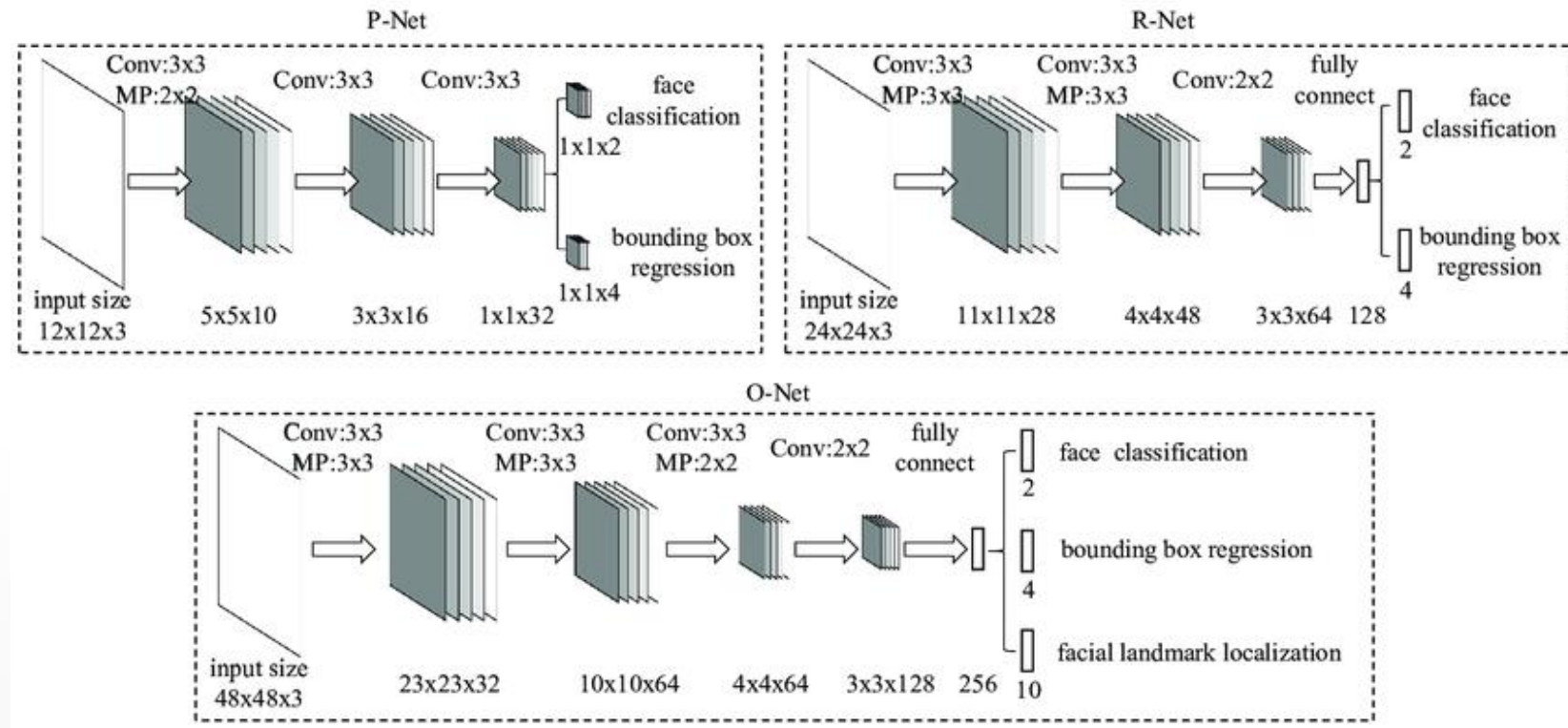
- There are perhaps four milestone systems on deep learning for face recognition that drove these innovations; they are: DeepFace, the DeepID series of systems, VGGFace, and FaceNet.
- DeepFace (2014) is a system based on deep convolutional neural networks, achieving near human-level performance on a standard benchmark dataset.
- The DeepID systems were among the first deep learning models to achieve better-than-human performance on the task, e.g., DeepID2 achieved 99.15% on the Labeled Faces in the Wild (LFW) dataset, which is better-than-human performance of 97.53%. Subsequent systems such as FaceNet and VGGFace improved upon these results.

- One of the more popular approaches is called Multi-Task Cascaded Convolutional Neural Network, or MTCNN (2018).
- The network uses a cascade structure with three CNNs:
 - First, the image is rescaled to a range of different sizes (called an image pyramid).
 - Then, the first model (Proposal Network or P-Net) proposes candidate facial regions (bounding boxes) at different scales.
 - The second model (Refine Network or R-Net) refines the bounding boxes by filtering false positives and regressing them, and
 - The third model (Output Network or O-Net) further refines the result and outputs five facial **landmarks: the eyes, nose and the corners of the mouth**.



Face Detection

- The model is called a multi-task network because each of the three models in the cascade (P-Net, R-Net and O-Net) are trained on three tasks, i.e., to make three types of predictions:
 - face classification,
 - bounding box regression, and
 - facial landmark localization.
- The three models are not connected directly; instead, outputs of the previous stage are fed as input to the next stage.
- This allows additional processing to be performed between stages; e.g., non-maximum suppression (NMS) is used to filter the candidate bounding boxes proposed by the first-stage P-Net prior to providing them to the second stage R-Net model.



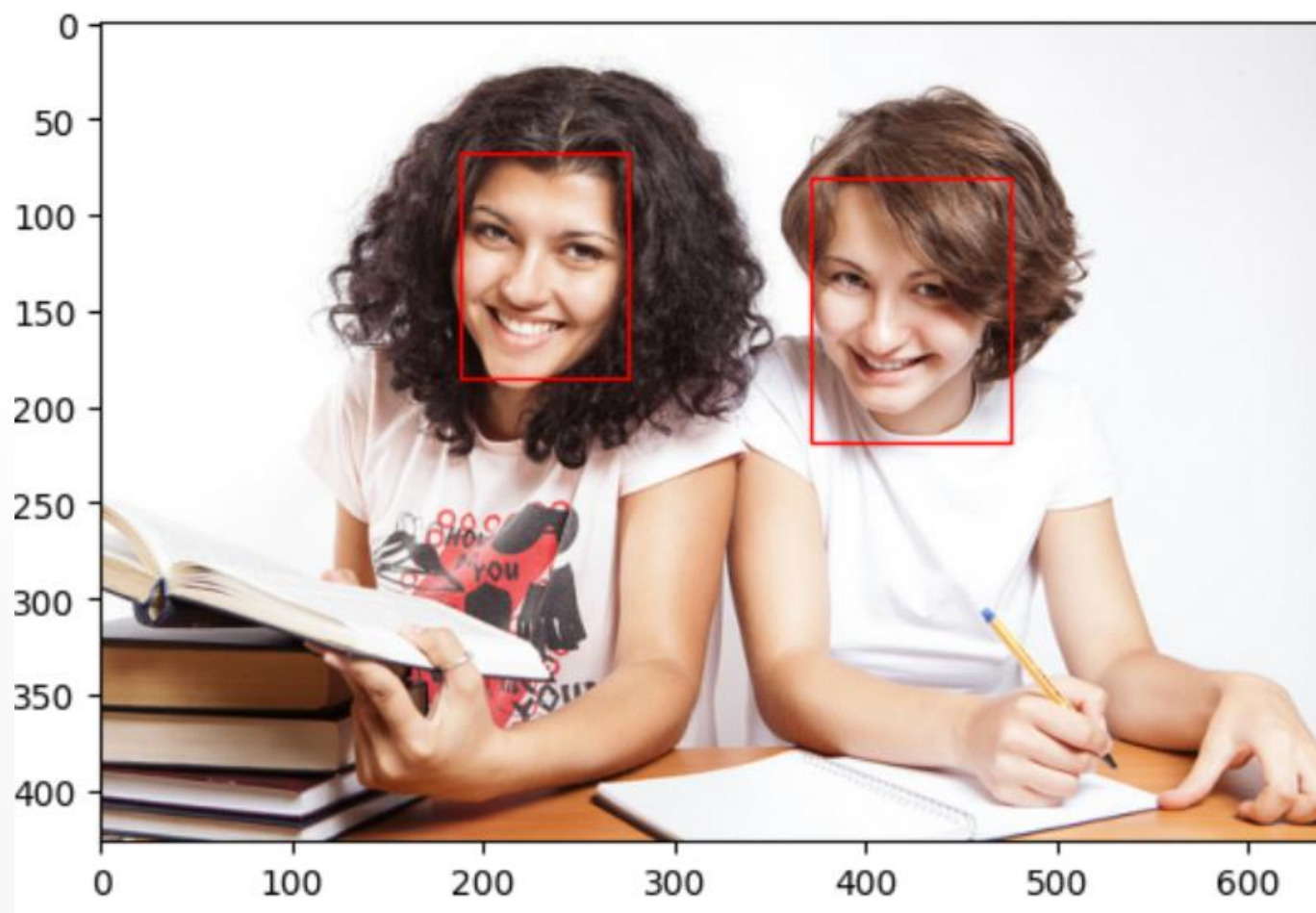
Face Detection Using MTCNN

```
{'box': [188, 67, 87, 118], 'confidence': np.float64(0.9973015189170837), 'keypoints': {'nose': [np.int64(219), np.int64(142)], 'mouth_right': [np.int64(245), np.int64(157)], 'right_eye': [np.int64(250), np.int64(118)], 'left_eye': [np.int64(208), np.int64(109)], 'mouth_left': [np.int64(200), np.int64(147)]}}
```

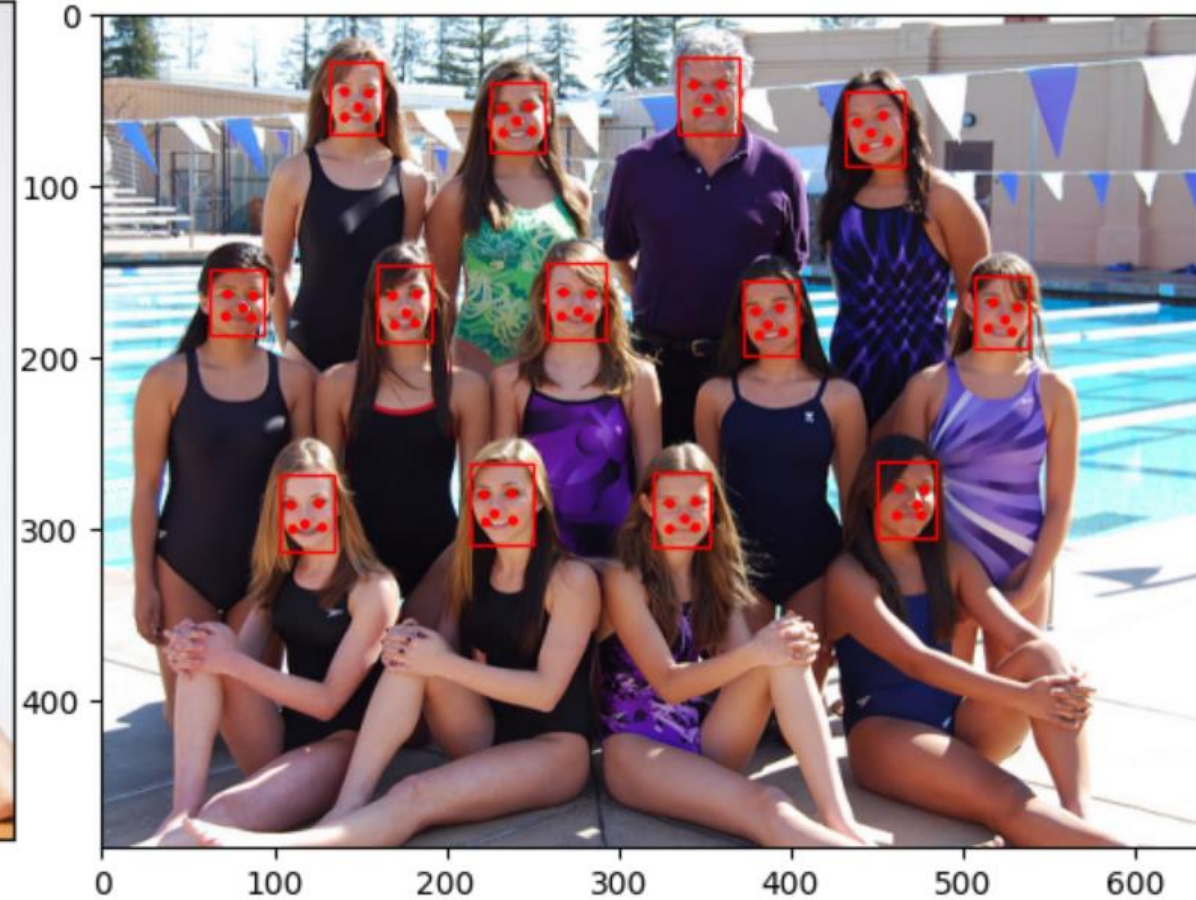
```
{'box': [371, 80, 105, 138], 'confidence': np.float64(0.9763906002044678), 'keypoints': {'nose': [np.int64(406), np.int64(166)], 'mouth_right': [np.int64(437), np.int64(187)], 'right_eye': [np.int64(440), np.int64(138)], 'left_eye': [np.int64(392), np.int64(134)], 'mouth_left': [np.int64(388), np.int64(184)]}}
```

```
# face detection with mtcnn on a photograph
from matplotlib import pyplot
from mtcnn.mtcnn import MTCNN
# Load image from file
filename = '../Data/face_detection_college_students.jpg'
pixels = pyplot.imread(filename)
# create the detector, using default weights
detector = MTCNN()
# detect faces in the image
faces = detector.detect_faces(pixels)
for face in faces:
    print(face)

filename = '../Data/face_detection_college_students.jpg'
# Load image from file
pixels = pyplot.imread(filename)
# create the detector, using default weights
detector = MTCNN()
# detect faces in the image
faces = detector.detect_faces(pixels)
# display faces on the original image
draw_image_with_boxes(filename, faces)
```



Face Detection Using MTCNN – Plot Landmarks

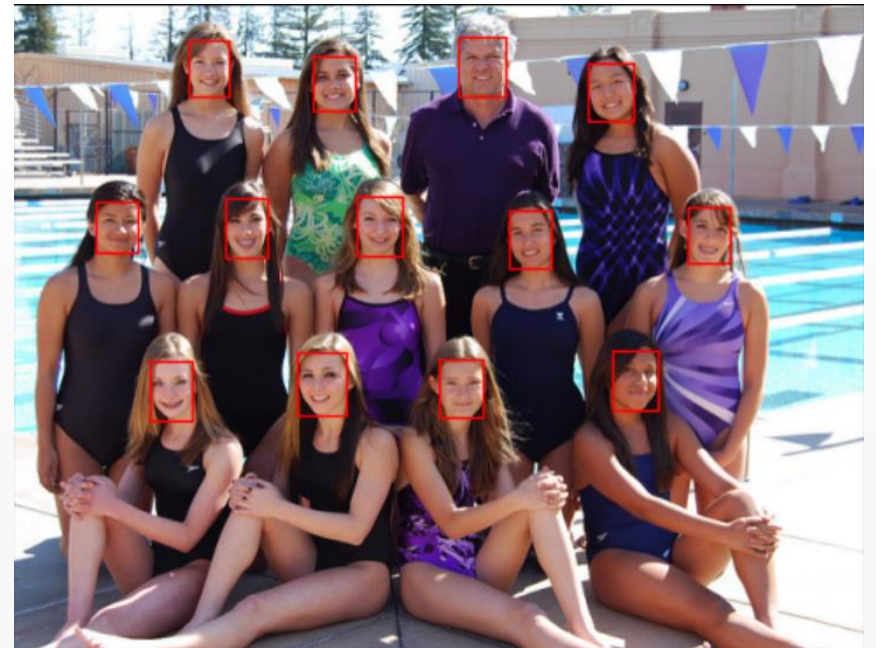
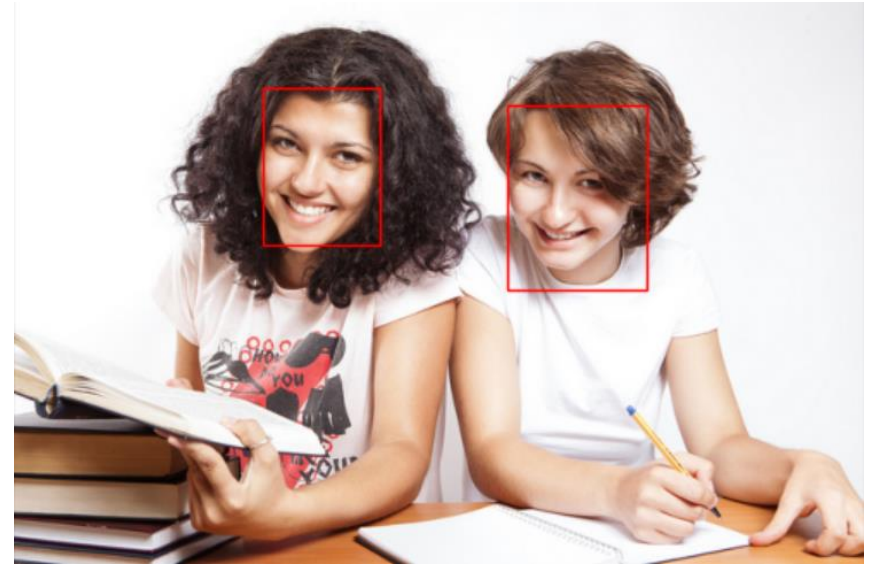


Face Detection Using MTCNN – Extract Faces

```
filename = '../Data/face_detection_swim_team.jpg'  
# load image from file  
pixels = pyplot.imread(filename)  
# create the detector, using default weights  
detector = MTCNN()  
# detect faces in the image  
faces = detector.detect_faces(pixels)  
# display faces on the original image  
draw_faces(filename, faces)
```



Face Detection Cascade Classifier vs MTCNN



Face Identification and Verification

Face Verification and Identification

- Face Verification. A one-to-one mapping of a given face against a known identity (e.g., is this the person?)
- Face Identification. A one-to-many mapping for a given face against a database of known faces (e.g., who is this person?)

Face Identification

- ResNet50 model used.
- Model expects input color images of faces with the shape of 244×244 and the output will be a class prediction of 8,631 people in the MS-Celeb-1M dataset.

```
# Load the photo and extract the face
pixels = extract_face('../Data/sharon_stone1.jpg')
# convert one face into samples
pixels = pixels.astype('float32')
samples = expand_dims(pixels, axis=0)
# prepare the face for the model, e.g. center pixels
samples = preprocess_input(samples, version=2)
# create a vggface model
model = VGGFace(model='resnet50')
# perform prediction
yhat = model.predict(samples)
# convert prediction into names
results = decode_predictions(yhat)
# display most likely results
for result in results[0]:
    print('%s: %.3f%' % (result[0][3:-1], result[1]*100))
```



```
Sharon_Stone: 99.572%
Noelle_Reno: 0.082%
Anita_Lipnicka: 0.033%
Elisabeth_R\xc3\xbf6hm: 0.027%
Emma_Atkins: 0.019%
```

Face Identification

```
# Load the photo and extract the face
pixels = extract_face('../Data/channing_tatum.jpg')
# convert one face into samples
pixels = pixels.astype('float32')
samples = expand_dims(pixels, axis=0)
# prepare the face for the model, e.g. center pixels
samples = preprocess_input(samples, version=2)
# create a vggface model
model = VGGFace(model='resnet50')
# perform prediction
yhat = model.predict(samples)
# convert prediction into names
results = decode_predictions(yhat)
# display most likely results
for result in results[0]:
    print('%s: %.3f%%' % (result[0][3:-1], result[1]*100))
```



Channing_Tatum: 93.826%
Nico_Rosberg: 0.173%
Eoghan_Quigg: 0.157%
Les_Miles: 0.122%
Nico_H\x3\xbc1kenberg: 0.091%

Face Verification with VGGFace2

- A VGGFace2 model can be used for face verification. This involves calculating a face embedding for a new given face and comparing the embedding to the embedding for the single example of the face known to the system.
- A face embedding is a vector that represents the features extracted from the face.
- This can then be compared with the vectors generated for other faces. For example, another vector that is close (by some measure) may be the same person, whereas another vector that is far (by some measure) may be a different person.
- Typical measures such as Euclidean distance and Cosine distance are calculated between two embeddings and faces are said to match or verify if the distance is below a predefined threshold, often tuned for a specific dataset or application.

Face Verification with VGGFace2

- First, we can load the VGGFace model without the classifier by setting the `include_top` argument to `False`, specifying the shape of the output via the `input_shape` and setting pooling to `avg` so that the filter maps at the output end of the model are reduced to a vector using global average pooling.

```
# create a vggface model  
model = VGGFace(model='resnet50', include_top=False, input_shape=(224, 224, 3), pooling='avg')
```

Face Verification with VGGFace2

```
# extract a single face from a given photograph
def extract_face(filename, required_size=(224, 224)):
    # Load image from file
    pixels = pyplot.imread(filename)
    # create the detector, using default weights
    detector = MTCNN()
    # detect faces in the image
    results = detector.detect_faces(pixels)
    # extract the bounding box from the first face
    x1, y1, width, height = results[0]['box']
    x2, y2 = x1 + width, y1 + height
    # extract the face
    face = pixels[y1:y2, x1:x2]
    # resize pixels to the model size
    image = Image.fromarray(face)
    image = image.resize(required_size)
    face_array = asarray(image)
    return face_array
```

```
# extract faces and calculate face embeddings for a list of photo files
def get_embeddings(fileNames):
    # extract faces
    faces = [extract_face(f) for f in fileNames]
    # convert into an array of samples
    samples = asarray(faces, 'float32')
    # prepare the face for the model, e.g. center pixels
    samples = preprocess_input(samples, version=2)
    # create a vggface model
    model = VGGFace(model='resnet50', include_top=False, input_shape=(224, 224, 3), pooling='avg')
    # perform prediction
    yhat = model.predict(samples)
    return yhat
```

```
# determine if a candidate face is a match for a known face
def is_match(known_embedding, candidate_embedding, thresh=0.5):
    # calculate distance between embeddings
    score = cosine(known_embedding, candidate_embedding)
    if score <= thresh:
        print('>face is a Match (%.3f <= %.3f)' % (score, thresh))
    else:
        print('>face is NOT a Match (%.3f > %.3f)' % (score, thresh))
```

Face Verification with VGGFace2



Anchor



Database

Positive Tests

>face is a Match (0.461 $\leq$ 0.500)

>face is a Match (0.312 $\leq$ 0.500)

Negative Tests

>face is NOT a Match (0.703 $>$ 0.500)

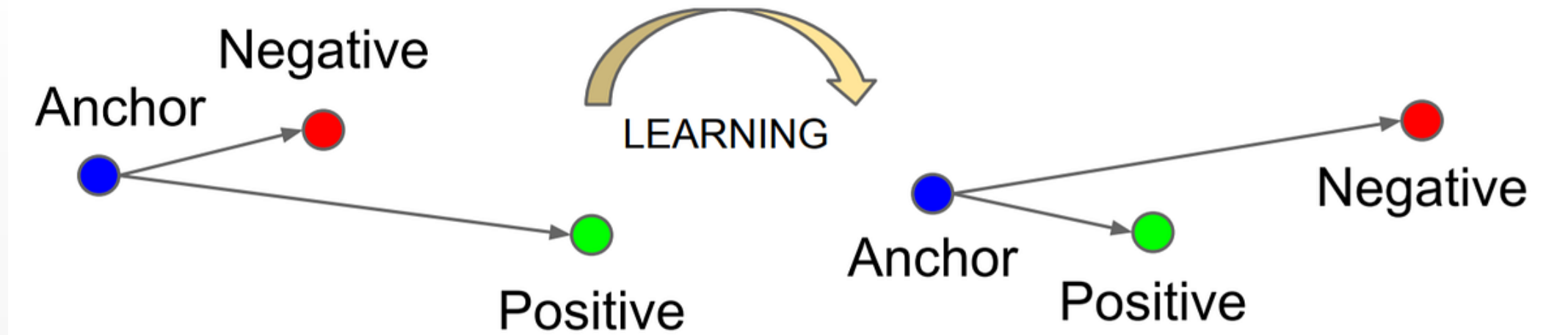
Face Classification with FaceNet

Face Classification with FaceNet

- The FaceNet system can be used to extract high-quality features from faces, called face embeddings, that can then be used to train a face identification system.
- We will develop a face detection system using FaceNet and an SVM classifier to identify people from photographs.
- Face detection is the process of automatically locating faces in a photograph and localizing them by drawing a bounding box around their extent.

FaceNet Model

- The model is a deep convolutional neural network trained via a triplet loss function that encourages vectors for the same identity to become more similar (smaller distance), whereas vectors for different identities are expected to become less similar (larger distance).
- The focus on training a model to create embeddings directly (rather than extracting them from an intermediate layer of a model) was an important innovation in this work.



Face Classification System

- The 5 Celebrity Faces Dataset is a small dataset that contains photographs of celebrities. It includes photos of: Ben Affleck, Elton John, Jerry Seinfeld, Madonna, and Mindy Kaling.

```
# specify folder to plot
folder = '../Data/5-celebrity-faces-dataset/train/ben_afflek/'
i = 1
# enumerate files
for filename in listdir(folder):
    # path
    path = folder + filename
    # get face
    face = extract_face(path)
    print(i, face.shape)
    # plot
    pyplot.subplot(2, 7, i)
    pyplot.axis('off')
    pyplot.imshow(face)
    i += 1
pyplot.show()
```



Face Classification System

- Detect faces in the photographs and reduce the dataset to a series of faces only.
- Create face embeddings.
 - A face embedding is a vector that represents the extracted features from the face.
 - This can then be compared with the vectors generated for other faces.
 - For example, another vector that is close (by some measure) may be the same person, whereas another vector that is far (by some measure) may be a different person.
 - The classifier model that we want to develop will take a face embedding as input and predict the identity of the face.
 - The FaceNet model will generate this embedding for a given image of a face.
 - The FaceNet model can be used as part of the classifier itself, or we can use the FaceNet model to preprocess a face to create a face embedding that can be stored and used as input to our classifier model. This latter approach is preferred as the FaceNet model is both large and slow to create a face embedding.
- Face embeddings are then normalized since distance metrics are used.
- Linear SVM is a common classifier when working with face embeddings.
- Evaluate the model.

```
Dataset: train=93, test=25  
Accuracy: train=100.000, test=100.000
```


Face Classification System

Predicted: jerry_seinfeld (87.431)

Expected: jerry_seinfeld

